Vector Indexing Techniques for AI Search Systems

A simple, plain-English guide to our project
Comparing Flat, IVF, and HNSW search on real data

Made for the team • Advanced Database Topics

1. What is this project?

Modern AI search (like semantic search, recommendations, and the retrieval step in chatbots) works by turning text into lists of numbers called **embeddings**. Sentences with similar meaning get similar numbers, so finding related text becomes a matter of finding the closest number-lists. When you have millions of these number-lists, searching them quickly is hard.

This project is a **benchmark** — a fair, controlled test — that compares the three most common ways of organising those number-lists for fast search, and measures which one is best for which situation.

2. What is the goal?

There is no single 'best' search method — each one trades off speed, accuracy, memory, and setup time differently. Our goal is to measure those trade-offs clearly so we can answer:

- How fast is each method, and how does speed change as data grows?
- How accurate is each method (does it find the truly closest results)?
- How much memory does each one use, and how long does it take to build?
- Which method should you pick, and when?

The three search methods (the heart of the project)

Imagine a huge library of sentences and you want the ones closest in meaning to your question. There are three ways to search:

Flat	Check every single item, one by one. Always 100% accurate, but slow.
IVF	Sort items into neighbourhoods, then search only the nearest few. Much faster, can miss a few.
HNSW	Build a web of shortcuts between items and hop along it. Very fast, can also miss a few.

Flat is the 'ground truth' — because it checks everything, its answers are always correct. We use it to measure how accurate the faster methods (IVF and HNSW) are.

3. Why we chose this dataset

We used the public **"A Million News Headlines"** dataset (ABC News, Australia). We picked it for three reasons:

- **It is real, meaningful text.** The project is about searching by meaning, so we need genuine sentences about real topics — politics, sport, weather, crime, and so on.
- **It is large.** With ~1.24 million headlines, we can sample any amount we want for testing. We used 20,000.
- **It is free and citable.** It is released under a public-domain (CC0) licence, so there are no usage restrictions.

4. What is inside the dataset?

It is a simple spreadsheet (CSV) with just two columns and about 1.24 million rows:

Column	Meaning	Example
publish_date	Date the article was published	20030219 (= 19 Feb 2003)
headline_text	The news headline itself	"act fire witnesses must be aware of defamation"

So it is simply ~1.24 million real news headlines written by journalists. We only use the headline text; the date is ignored. Our helper script samples 20,000 of them into a clean list that the benchmark reads.

5. The code: what each file does

The project is split into small pieces, each doing one job. Together they form a pipeline: load text → turn into numbers → build search indexes → run searches → measure and chart the results.

File	What it does
data_ingestion.py	Loads and cleans the text (or generates synthetic test data).
embeddings.py	Turns each sentence into a list of numbers (meaning) using Sentence-BERT.
indexing.py	Builds the three search structures: Flat, IVF, and HNSW.
query_engine.py	Runs the searches, times them, and checks how accurate they were.
visualization.py	Draws the charts and saves the results tables.
benchmark.py	The 'conductor' — runs all the steps above in the right order.
run_benchmark.py	The button you press; reads a config file and starts a run.
prepare_headlines.py	Turns the big 1.24M-row CSV into a clean list of 20,000 headlines.

The most important functions

- **encode(texts)** (embeddings.py) — turns a batch of sentences into number-lists.
- **build(vectors)** (indexing.py) — builds a Flat, IVF, or HNSW index and times how long it takes plus how much memory it uses.
- **run_queries(index, queries, k)** (query_engine.py) — searches for the top-k closest items and records the time for each query.
- **recall_at_k(approx, ground_truth, k)** (query_engine.py) — accuracy check: of the truly-closest k results (from Flat), how many did the fast method find?
- **run_single_scale / run_scalability** (benchmark.py) — run the whole test once, or repeat it at growing dataset sizes.

6. The output: what we measured

For every method, the benchmark records four numbers:

- **Query latency** — how long one search takes, in milliseconds. Lower is better.
- **Recall@10** — out of the 10 truly-closest results, how many were found. Higher is better (Flat is always 1.000).
- **Memory** — how much space the search index takes, in megabytes. Lower is better.
- **Build time** — how long it takes to set the index up, in seconds. Lower is better.

Result 1 — Real data (20,000 news headlines)

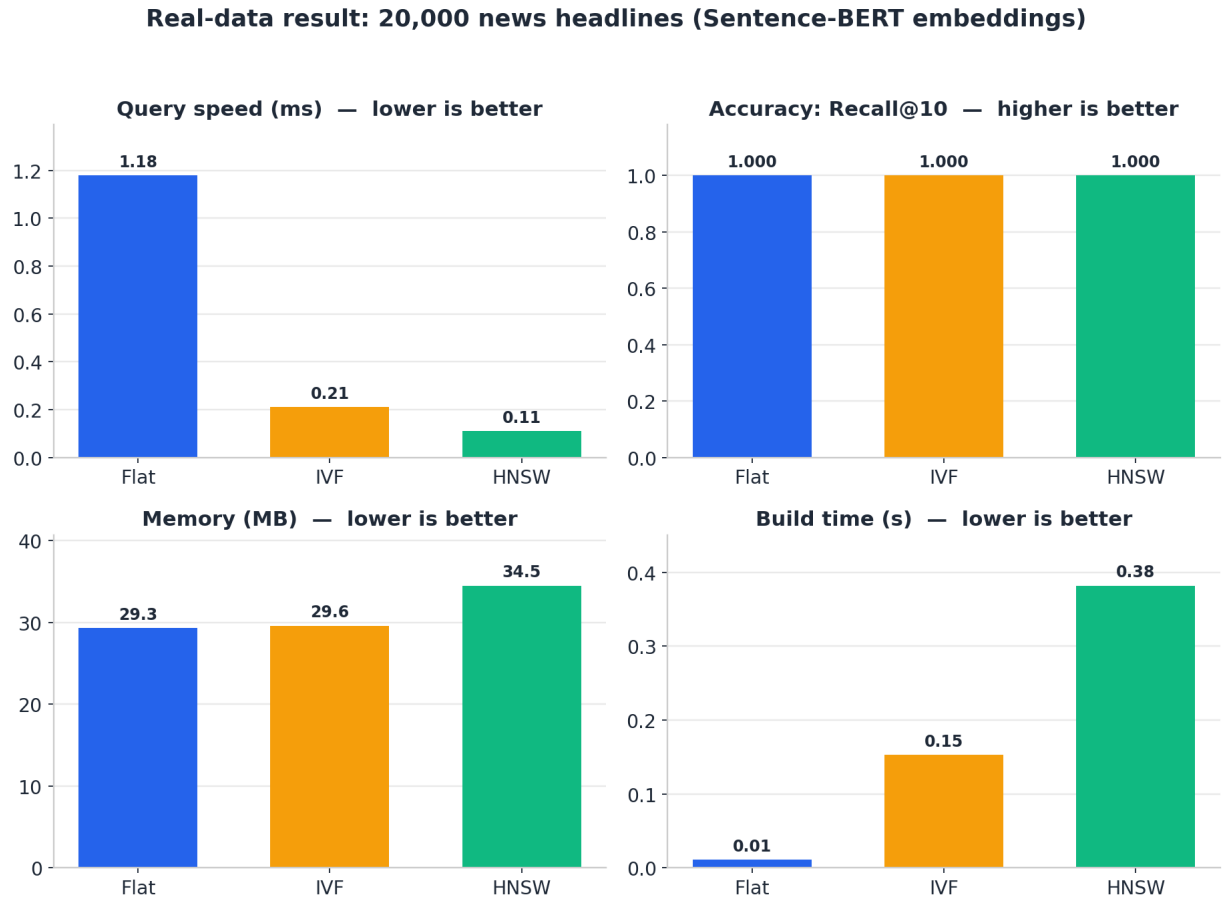

Figure 1: The four measurements on real news headlines. All three methods reach the same accuracy (recall 1.000), but IVF and HNSW are much faster than Flat - HNSW is fastest here, while Flat costs the least to build and memory.

Method	Latency (ms)	Recall@10	Memory (MB)	Build (s)
Flat	1.18	1.000	29.3	0.01
IVF	0.21	1.000	29.6	0.15
HNSW	0.11	1.000	34.5	0.38

Result 2 — Scalability (how things change as data grows)

Figure 2: Each measurement as the dataset grows from 1,000 to 100,000 vectors. Flat's search time climbs steeply; IVF and HNSW stay fast. But HNSW's accuracy slowly drops as data grows.

Result 3 — Parameter tuning (the speed/accuracy knob)

IVF and HNSW each have a knob that trades speed for accuracy: IVF's **nprobe** (how many clusters it searches) and HNSW's **efSearch** (how wide the graph search is). We swept both on the real data:

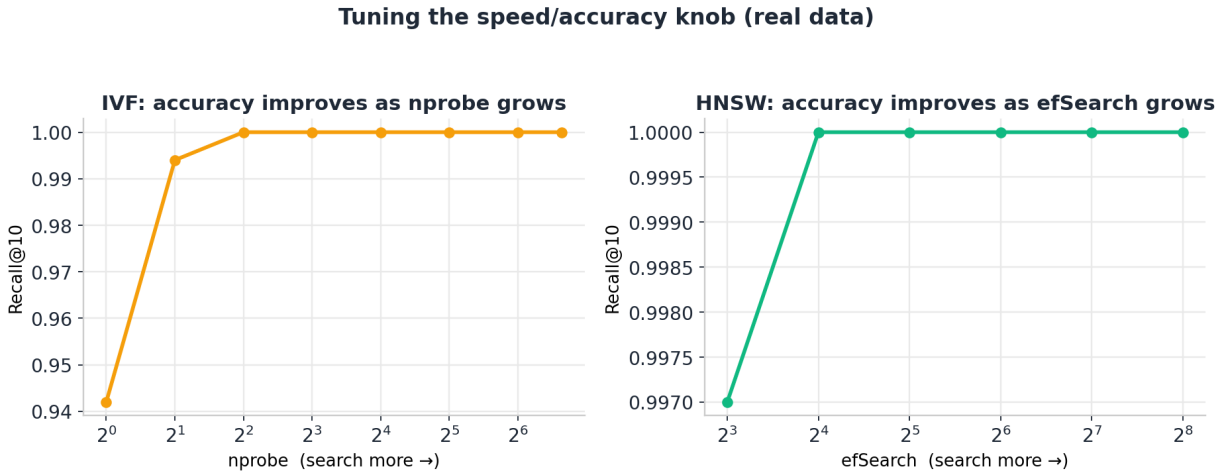

Figure 3: Accuracy rises as each knob increases, then flattens. IVF reaches 1.000 by nprobe=4; HNSW by efSearch=16.

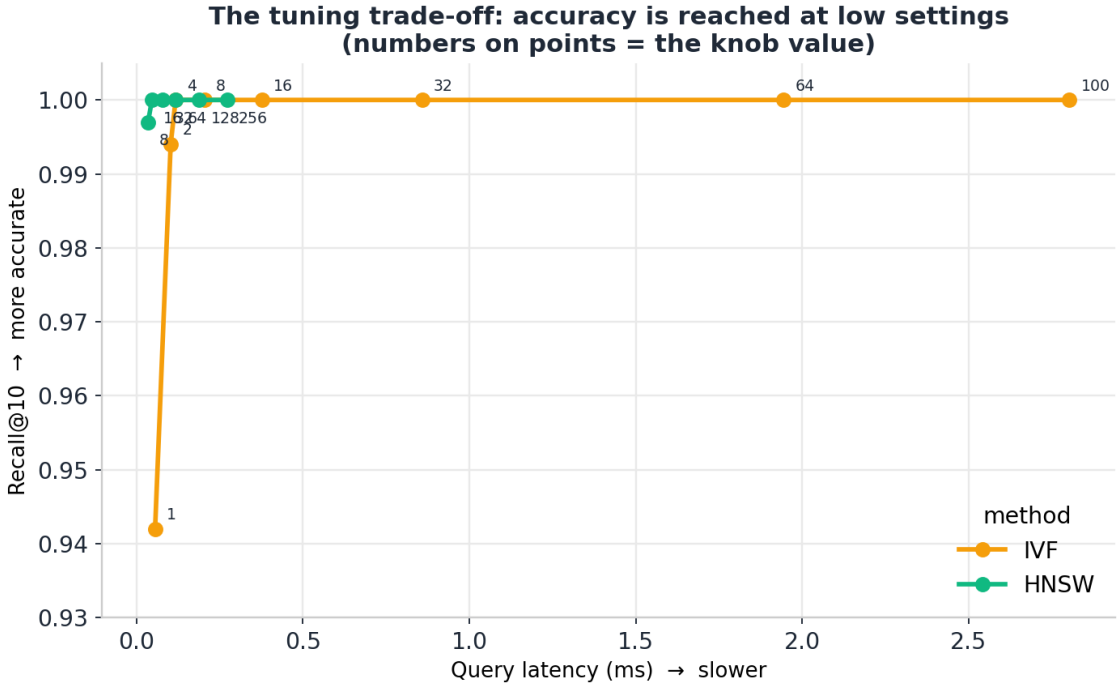

Figure 4: The trade-off curve. Both methods reach full accuracy at low settings; searching harder beyond that only adds latency (numbers on points = the knob value).

7. What we found (key takeaways)

- **Flat doesn't scale.** Its search time grew almost in a straight line with the data — from 0.07 ms at 1K to 8.2 ms at 100K. Perfectly accurate, but too slow for large data.

- **IVF and HNSW are far faster.** On the real headlines both matched Flat's accuracy (recall 1.000) while being 5-10x faster, and both stayed well under 1 ms even at 100K vectors.
- **Accuracy is reached at low settings.** The tuning sweep showed IVF needs only nprobe=4 and HNSW only efSearch=16 to hit full recall on this data. Searching harder beyond that just wastes time.
- **But settings must scale with the data.** In the scalability test (fixed settings, growing data), HNSW's recall slowly dropped from 1.00 to 0.70 as the dataset grew to 100K — a reminder that the knob has to be raised as data grows.
- **Every method has a cost.** HNSW is fastest at query time but slowest to build and uses the most memory; Flat is cheapest to build but slowest to search; IVF sits in between.

Bottom line: there is no universal winner. The right index, and the right settings for it, depend on your dataset size, how accurate you need to be, and whether you can afford the build cost and memory. Measuring those trade-offs fairly is the main result of the project.

8. How to run it (quick reference)

All commands run from the **vecbench** folder:

```
pip install -r requirements.txt
pip install sentence-transformers

# 1) prepare the dataset (one time)
python prepare_headlines.py --csv abcnews-date-text.csv --out data/headlines.txt --n
20000

# 2) scalability test (synthetic, no download)
python run_benchmark.py --config configs/scalability.yaml

# 3) real-data test (news headlines + Sentence-BERT)
python run_benchmark.py --config configs/text_sbert.yaml
```

Results (tables + charts) appear in the **results/** folder.